

---

# Propozycje zastosowania LLM i RAG w projekcie Eden

---

Data: 2026-05-12

---

## Przegląd

Aktualny stack (Semgrep + metryki + estymator oparty na medianie) działa dobrze dla przypadków, które można opisać regułami lub statystykami. LLM/RAG dają wartość w dwóch miejscach, gdzie reguły i statystyki zawodzą:

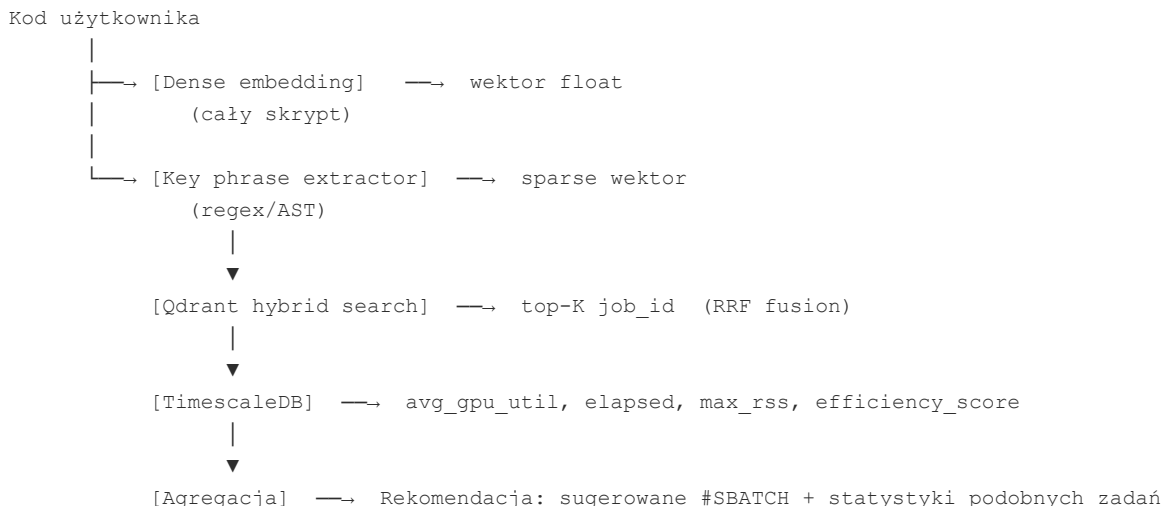
- **Szacowanie zasobów przez podobieństwo kodu** — hybrid search znajduje historycznie podobne zadania i zwraca ile zasobów faktycznie zużyły
  - **Wyjaśnianie rekomendacji** — LLM tłumaczy *dlaczego* ten kod potrzebuje tyle zasobów, edukując użytkowników z tła matematycznego w zakresie HPC
- 

## 1. RAG dla estymatora zasobów

### Idea

Użytkownik przesyła skrypt przed uruchomieniem. System embedduje kod, przeszukuje bazę historycznych zadań, odnajduje semantycznie podobny kod i zwraca ile zasobów tamte zadania faktycznie zużyły. Użytkownik widzi: *"Twój kod jest podobny do 3 zadań z ostatnich 6 miesięcy. Zająły średnio 2.4h, 18GB VRAM, efektywność GPU 71%."*

### Pipeline



## Strategia retrieval: hybrid search

Czyste wyszukiwanie wektorowe działa dobrze na podobieństwo semantyczne, ale kod ma specyficzną cechę: dwa skrypty mogą być semantycznie bliskie, a jednocześnie różnić się kluczowymi szczegółami (np. jeden używa `torch.nn.DataParallel`, drugi `DistributedDataParallel`). Hybrid search łączy dwa sygnały:

- **Dense** — embedding całego skryptu, łapie ogólny charakter zadania (training vs inference, rodzaj modelu)
- **Sparse** — nie zwykłe słowa kluczowe, ale **key phrases**: importy bibliotek, wywołania GPU API, parametry `#SBATCH`

Przykładowe key phrases:

```
import torch
model.cuda()
DataLoader(num_workers=8)
#SBATCH --gres=gpu:2
torch.nn.parallel.DistributedDataParallel
```

## Co embedować

Cały skrypt = jeden wektor, bez chunkowania. Skrypt Slurm jest z reguły krótki, embedding całości nie traci sygnału i upraszcza retrieval.

## Jakie przykładowe key phrases ekstrahować

```
KEY_PHRASE_PATTERNS = [
    r"^import\s+\S+",          # importy bibliotek
    r"^from\s+\S+\s+import",   # from-import
    r"\.\s*cuda\(\)",          # przeniesienie na GPU
    r"device\s*=\s*['\"]cuda[ '\" ]", # device='cuda'
    r"DataLoader\s*\(",        # DataLoader (num_workers, pin_memory)
    r"#SBATCH\s+--\S+",        # parametry SBATCH
    r"torch\.nn\.w+Parallel",  # strategię równoległości
    r"amp\.autocast|GradScaler", # mixed precision
    r"torch\.utils\.checkpoint", # gradient checkpointing
    r"\.to\(\s*device\s*\)|\.to\(['\"]cuda", # jawne przeniesienie tensora
]
```

## Tech stack

| Komponent            | Rekomendacja                   | Powód                                                     |
|----------------------|--------------------------------|-----------------------------------------------------------|
| Embedding (dense)    | <code>BAAI/bge-small-en</code> | Lokalny, prywatny, dobry na kod                           |
| Sparse (key phrases) | własny ekstraktor regex/AST    | Pełna kontrola nad tym co jest "słowem kluczowym"         |
| Vector store         | <b>Qdrant</b>                  | Natywny hybrid search (dense + sparse w jednym zapytaniu) |

|           |                                       |                                                           |
|-----------|---------------------------------------|-----------------------------------------------------------|
| Fusion    | <code>alpha</code> w Qdrant Query API | Qdrant łączy oba wyniki wewnętrznie, bez dodatkowego kodu |
| Generacja | któryś model Anthropic                | Są najlepsze                                              |

## 2. LLM jako wyjaśnienie rekomendacji zasobów

### Kontekst i problem

Sekcja 1 zwraca rekomendację w postaci liczb: *"podobne zadania potrzebowały 18GB VRAM, 2 GPU, czas 2.4h"*. Użytkownik nietechniczny widzi te liczby, ale nie rozumie *dlaczego* jego kod wymaga tyle zasobów — i przez to albo ignoruje rekomendację, albo nie wie jak zoptymalizować skrypt żeby potrzebował mniej.

LLM wypełnia tę lukę. Wejście to rekomendacja z sekcji 1 plus wyniki Semgrep (który wykrywa konkretne wzorce w kodzie wpływające na zużycie zasobów). Wyjście to wyjaśnienie skrojone pod odbiorców bez tła HPC.

Przykład: Semgrep wykrywa transfer CPU→GPU w pętli. LLM tłumaczy że to powód, dla którego GPU jest zajęte tylko 38% czasu (co widać w `avg_gpu_util` z historycznych zadań), a tym samym dlaczego zadanie potrzebuje więcej czasu niż mogłoby. Użytkownik rozumie połączenie między kodem a metrykami.

### Cel edukacyjny

LLM pełni tutaj rolę nie tylko asystenta do naprawy kodu, ale **tutora HPC**. Każde wykryte przez Semgrep naruszenie jest okazją, żeby wyjaśnić użytkownikowi jeden konkretny mechanizm działania GPU lub klastra. Użytkownicy, którzy regularnie otrzymują takie wyjaśnienia, z czasem przestają popełniać te same błędy — zmniejsza się liczba zadań z niską efektywnością.

### LLM jako post-processor

Po uruchomieniu Semgrep na kodzie użytkownika wyniki JSON trafiają do LLM razem z fragmentem kodu. Prompt jest skrojony pod odbiorców bez tła HPC:

```
Oto wyniki analizy statycznej kodu HPC:
[JSON z Semgrep]
```

```
Oto odpowiedni fragment kodu:
[linie 40-45 z pliku]
```

```
Wyjaśnij po polsku, zakładając że odbiorca jest bardzo inteligentny, ale nie zna wzorców HPC i programowania
1. Co fizycznie dzieje się w GPU gdy uruchamia się ten kod (dlaczego jest wolno)
2. Jaką konkretną zmianę wprowadzić (pokaż before/after)
3. Skąd bierze się przyspieszenie – intuicja, nie tylko liczba
```

Przykład wyjścia dla reguły `cpu-gpu-transfer-in-loop` :

> **Linia 42 — transfer CPU→GPU w pętli** > > Twój kod tworzy tensor na CPU ( `torch.tensor(x)` ) i dopiero potem przenosi go na GPU ( `.cuda()` ). GPU i CPU to dwa oddzielne układy z oddzielną pamięcią — każdy transfer przez magistralę PCIe ma stały narzut rzędu 50–200  $\mu$ s, niezależnie od rozmiaru danych. Przy 1000 iteracjach pętli sumarycznie tracisz do 0.2 sekundy tylko na przesyłanie — zanim GPU w ogóle zacznie liczyć. > > **Zmiana:** > ``python > # Przed > for x in data: > t = torch.tensor(x).cuda() > > # Po > for x in data: > t = torch.tensor(x, device='cuda') > `` > GPU otrzymuje dane bezpośrednio w swojej pamięci, bez pośredniego etapu na CPU. Przy małych tensorach różnica może być 3–10x dla samej pętli.

## Kategorie reguł i co LLM wyjaśnia

Każda reguła Semgrep należy do kategorii — LLM dostosowuje poziom wyjaśnienia:

| Kategoria reguły                                                       | Co LLM tłumaczy                                           |
|------------------------------------------------------------------------|-----------------------------------------------------------|
| Transfer CPU↔GPU                                                       | Architektura pamięci GPU vs CPU, koszt PCIe               |
| Synchronizacja ( <code>.item()</code> , <code>.numpy()</code> w pętli) | Dlaczego GPU pipeline jest asynchroniczny                 |
| Alokacja pamięci w pętli                                               | VRAM fragmentation, GC overhead                           |
| Brak <code>pin_memory</code> / <code>num_workers</code>                | Jak DataLoader nakłada się z GPU kernelami                |
| Over-provisioning GPU                                                  | Koszt dla innych użytkowników klastra, scheduler fairness |
| Brak mixed precision                                                   | Czym jest FP16/BF16, kiedy nie traci się precyzji         |

## Dwa tryby odpowiedzi

Nie każdy użytkownik chce pełnego wykładu. LLM generuje odpowiedź w dwóch blokach:

[KRÓTKO] Przenosisz dane CPU→GPU w pętli. Użyj `device='cuda'` przy tworzeniu tensora. ~3x szybciej.

[WYJAŚNIENIE] GPU i CPU mają oddzielną pamięć...

API zwraca oba. Frontend pokazuje tylko krótkie podsumowanie z opcją rozwinięcia — użytkownik który już rozumie problem klika "pomiń", nowy użytkownik czyta wyjaśnienie.

## Integracja z obecnym pipeline

Semgrep już jest wybrany jako narzędzie analizy statycznej. LLM to dodatkowa warstwa — nie zmienia reguł ani decyzji o dopuszczeniu zadania do kolejki, tylko wzbogaca feedback:

```
sbatch script.py
  → Semgrep scan (obowiązkowe)
  → LLM explanation
  → Wyświetl użytkownikowi przed potwierdzeniem submity
  → Queue job
```

Użytkownik widzi ostrzeżenia z wyjaśnieniami zanim zdecyduje czy kontynuować. To nie blokuje — edukuje.

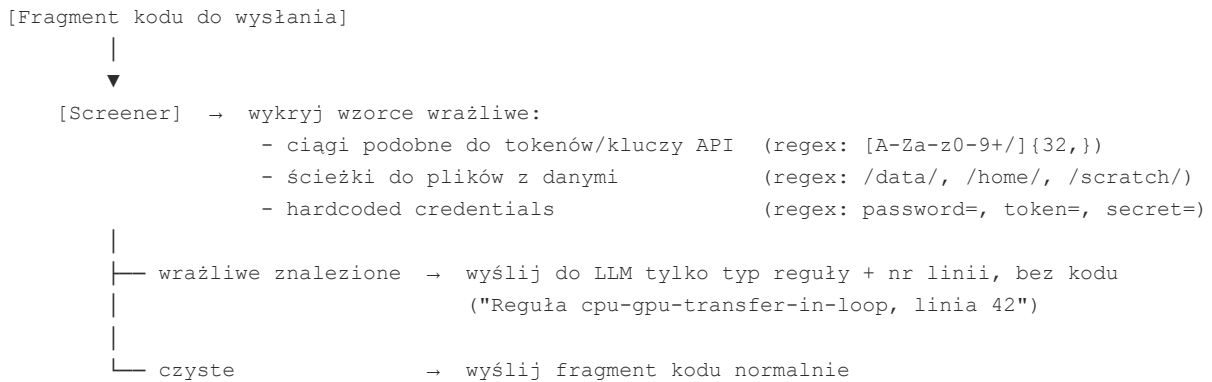
---

## Uwagi o prywatności

**RAG estimator (sekcja 1):** architektura jest w pełni lokalna — dense embedding i sparse ekstrakcja w `rag-advisor`, Qdrant w sieci `eden-net`. Zewnętrzne API (potrzebne do wygenerowania odpowiedzi w postaci *"podobne zadania potrzebowały 18GB VRAM, 2 GPU, czas 2.4h"*) dostaje tylko zagregowane metryki z TimescaleDB, nie surowy kod.

**LLM explanation dla Semgrep (sekcja 2):** fragment kodu (~10 linii wokół wykrytego problemu) trafia do zewnętrznego API. To jest ekspozycja danych — kod może zawierać ścieżki, nazwy zmiennych, tokeny zakodowane na sztywno, logikę biznesową.

Przed wysłaniem fragmentu do API wymagany jest screening:



Screeener to prosta funkcja regex, nie wymaga osobnego serwisu. Gdy fragment jest zablokowany, LLM i tak może wygenerować generyczne wyjaśnienie reguły — mniej precyzyjne, ale bez ryzyka wycieku.